

Rising Waters

Flood Management System — AI/ML Project

Code-Layout, Readability and Reusability

Phase 5: Project Development Phase

IBM Cloud

XGBoost 96.55%

Skill Wallet

AI/ML Track

Project Directory Structure

The project follows a modular, layered directory structure with separation of concerns:

<code>app.py</code>	Main Flask controller — routes, session, logging
<code>model/train_model.py</code>	ML pipeline: preprocessing, training, evaluation, save
<code>model/generate_dataset.py</code>	Synthetic dataset generator (5,000 rows)
<code>model/floods.save</code>	Serialized XGBoost model (joblib)
<code>model/scaler.save</code>	Fitted StandardScaler (joblib)
<code>utils/preprocessing.py</code>	Feature names, validation, scaling helper
<code>templates/base.html</code>	Shared layout (nav, footer, CSS imports)
<code>templates/result_flood.html</code>	Dedicated Flood Chance result page
<code>templates/result_noflood.html</code>	Dedicated No Flood Chance result page
<code>static/css/style.css</code>	Premium dark-theme CSS (glassmorphism)

Code Quality Standards

- All Python files include module-level docstrings with purpose description
- Functions are single-responsibility, each under 40 lines
- Constants defined at module level (FEATURE_NAMES, MODEL_PATH)
- HTML templates extend base.html — zero code duplication
- CSS uses CSS custom properties (variables) for consistent theming
- All routes return meaningful HTTP status codes